

Lexical Analysis

1. Introduction to Lexical Analysis

1.1 What is Lexical Analysis?

Lexical Analysis is the first phase of a compiler. It is the process of converting a sequence of characters (source code) into a sequence of tokens.

Simple Definition: Breaking down source code into meaningful pieces called tokens.

Alternative Name: Lexical Analyzer is also called a "**Scanner**" or "**Tokenizer**"

1.2 Basic Function

The lexical analyzer takes raw source code as input and produces tokens as output.

Example Input (Source Code):

```
int x = y + 10;
```

Example Output (Tokens):

TOKEN 1: <KEYWORD, int>

TOKEN 2: <IDENTIFIER, x>

TOKEN 3: <OPERATOR, =>

TOKEN 4: <IDENTIFIER, y>

TOKEN 5: <OPERATOR, +>

TOKEN 6: <NUMBER, 10>

TOKEN 7: <DELIMITER, ;>

2. Why is Lexical Analysis Needed?

2.1 Key Reasons

Reason 1: Simplification

- Breaks complex source code into simple, manageable pieces
- Makes the next phase (parsing) easier

Reason 2: Removes Unnecessary Elements

- Eliminates whitespace (spaces, tabs, newlines)
- Removes comments from the code
- These are not needed for compilation

Reason 3: Speed and Efficiency

- Tokens are easier to process than raw characters
- Parser works faster with tokens

Reason 4: Error Detection

- Catches invalid characters early in compilation
- Reports lexical errors before parsing begins

2.2 Example: Whitespace Handling

Input Code:

```
int  x  =  20  ;
```

(Notice multiple spaces)

Scanner's Action:

- Ignores all extra spaces
- Produces clean token stream

Output Tokens:

<KEYWORD, int>

<IDENTIFIER, x>

<OPERATOR, =>

<NUMBER, 20>

<DELIMITER, ;>

The multiple spaces are completely removed!

3. Basic Concepts and Terminology

3.1 Lexeme

Definition: A lexeme is the actual sequence of characters (substring) in the source code that matches a pattern.

Simple Explanation: The actual text from your code.

Examples:

Lexeme What it is

"while" Keyword lexeme

"x" Identifier lexeme

"123" Number lexeme

"=" Operator lexeme

";" Delimiter lexeme

3.2 Token

Definition: A token is a category or class that represents a type of lexeme.

Simple Explanation: The classification or label for the lexeme.

Token Format:

<token-name, attribute-value>

Components:

- **token-name:** The category (IDENTIFIER, NUMBER, KEYWORD, etc.)
- **attribute-value:** The actual value or additional information

Examples:

Lexeme Token Representation

"int" <KEYWORD, int>

"x" <IDENTIFIER, x>

"123" <NUMBER, 123>

"=" <OPERATOR, =>

";" <DELIMITER, ;>

3.3 Pattern

Definition: A pattern is a rule or description that defines all possible lexemes for a particular token type.

Simple Explanation: The rule that describes what a valid lexeme looks like.

Patterns are written using Regular Expressions.

Examples:

Pattern 1: IDENTIFIER

Pattern: letter (letter | digit)*

Meaning: Start with a letter, followed by zero or more letters or digits

Valid: x, count, var1, myVariable

Invalid: 1x, _var, 123

Pattern 2: NUMBER (Integer)

Pattern: digit+

Meaning: One or more digits

Valid: 0, 123, 9999

Invalid: 12.5, abc, 1x

Pattern 3: KEYWORD (for "while")

Pattern: while

Meaning: Exact match of the string "while"

Valid: while

Invalid: While, WHILE, wh1e

Pattern 4: OPERATOR (for +)

Pattern: +

Meaning: The plus symbol

Valid: +

Invalid: ++, +=

3.4 Relationship Between Lexeme, Token, and Pattern

Visual Representation:

SOURCE CODE:	int x = 20;				
LEXEME:	"int"	"x"	"="	"20"	";"
	↓	↓	↓	↓	↓
PATTERN:	keyword (exact)	identifier letter(l d)*	operator operator	number digit+	delimiter specific
	↓	↓	↓	↓	↓
TOKEN:	KEYWORD <KW,int>	IDENTIFIER <ID,x>	OPERATOR <OP,=>	NUMBER <NUM,20>	DELIMITER <DELIM,;>

4. What Does the Lexical Analyzer Do Internally?

4.1 Input and Output

INPUT: Raw program characters (source code as text)

OUTPUT: Token stream for the parser

Flow Diagram:

```
SOURCE CODE → [LEXICAL ANALYZER] → TOKEN STREAM → PARSER
   (text)           (Scanner)           (tokens)      (Syntax Analysis)
```

4.2 Internal Working Process

The scanner performs these steps:

Step 1: Read Characters

- Reads source code character by character
- Processes from left to right

Step 2: Group Characters

- Groups sequences of characters into lexemes
- Identifies where one lexeme ends and another begins

Step 3: Pattern Matching

- Matches grouped characters against patterns
- Uses regular expressions for pattern matching

Step 4: Produce Tokens

- Creates tokens for recognized lexemes
- Attaches appropriate token type and value

Step 5: Ignore Whitespace and Comments

- Skips spaces, tabs, newlines
- Removes comments (single-line `//` and multi-line `/* */`)

Step 6: Report Errors

- Detects invalid characters
- Reports lexical errors with line numbers

4.3 Detailed Example

Source Code:

```
int count = 5;
```

Character-by-Character Processing:

Position 1-3: Read 'i', 'n', 't'

→ Check pattern: matches keyword "int"

→ Create token: <KEYWORD, int>

Position 4: Read ' ' (space)

→ Ignore whitespace

→ Continue

Position 5-9: Read 'c', 'o', 'u', 'n', 't'

→ Check pattern: matches identifier pattern [a-z]+

→ Create token: <IDENTIFIER, count>

Position 10: Read ' ' (space)

→ Ignore whitespace

→ Continue

Position 11: Read '='

→ Check pattern: matches operator "="

→ Create token: <OPERATOR, =>

Position 12: Read ' ' (space)

→ Ignore whitespace

→ Continue

Position 13: Read '5'

→ Check pattern: matches number pattern [0-9]+

→ Create token: <NUMBER, 5>

Position 14: Read ';'

→ Check pattern: matches delimiter ";"

→ Create token: <DELIMITER, ;>

Final Token Stream:

<KEYWORD, int>

<IDENTIFIER, count>

<OPERATOR, =>

<NUMBER, 5>

<DELIMITER, ;>

5. Finite Automata in Lexical Analysis

5.1 Role of Finite Automata

Lexical patterns are described using **Regular Expressions**, but they are implemented using **Finite Automata**.

Process Flow:

STEP 1: Write patterns as Regular Expressions



STEP 2: Convert Regular Expression to NFA



STEP 3: Convert NFA to DFA



STEP 4: Minimize the DFA (optimize)



STEP 5: Use minimized DFA for scanning

5.2 Example Regular Expressions for Tokens

Pattern 1: Identifier

Regular Expression: `[a-zA-Z][a-zA-Z0-9]*`

Meaning: Letter followed by any number of letters or digits

Examples: x, count, myVar123

Pattern 2: Integer Number

Regular Expression: `[0-9]+`

Meaning: One or more digits

Examples: 0, 123, 9999

Pattern 3: Floating Point Number

Regular Expression: $[0-9]^+\.[0-9]^+$

Meaning: Digits, decimal point, then more digits

Examples: 3.14, 0.5, 123.456

Pattern 4: Keyword (while)

Regular Expression: while

Meaning: Exact string "while"

Example: while

5.3 Why Use Finite Automata?

Reason 1: Efficiency

- DFAs can process input in linear time
- $O(n)$ where n is the length of input

Reason 2: Deterministic

- DFA always knows the next state
- No backtracking needed

Reason 3: Easy to Implement

- Can be coded as a simple state transition table
- Fast execution in hardware or software

6. Example DFA for Identifiers

6.1 Pattern Definition

Identifier Pattern: letter (letter | digit)*

This means:

- Must start with a letter (a-z, A-Z)
- Can be followed by any number of letters or digits

6.2 DFA Structure

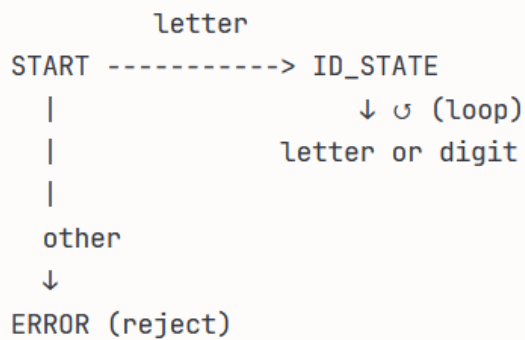
States:

- **START:** Initial state
- **ID_STATE:** Accepting state (valid identifier recognized)
- **ERROR:** Reject state

Transitions:

State Diagram (Text Representation):

State Diagram (Text Representation):



Detailed Description:

FROM START:

- If input is a letter → go to ID_STATE
- If input is anything else → go to ERROR

FROM ID STATE:

- If input is a letter → stay in ID_STATE
- If input is a digit → stay in ID_STATE
- If input is anything else → end of identifier (accept)

6.3 Example Execution

Example 1: Input "count"

```
Step 1: Read 'c' (letter)
  Current State: START
  Action: Transition to ID_STATE

Step 2: Read 'o' (letter)
  Current State: ID_STATE
  Action: Stay in ID_STATE

Step 3: Read 'u' (letter)
  Current State: ID_STATE
  Action: Stay in ID_STATE

Step 4: Read 'n' (letter)
  Current State: ID_STATE
  Action: Stay in ID_STATE

Step 5: Read 't' (letter)
  Current State: ID_STATE
  Action: Stay in ID_STATE

Step 6: Read ' ' (space - not letter/digit)
  Current State: ID_STATE
  Action: End of identifier, ACCEPT
```

Result: "count" is recognized as IDENTIFIER

Example 2: Input "var123"

```
Step 1: Read 'v' (letter) → ID_STATE
Step 2: Read 'a' (letter) → ID_STATE
Step 3: Read 'r' (letter) → ID_STATE
Step 4: Read '1' (digit) → ID_STATE
Step 5: Read '2' (digit) → ID_STATE
Step 6: Read '3' (digit) → ID_STATE
Step 7: Read ' ' (space) → END, ACCEPT
```

Result: "var123" is recognized as IDENTIFIER

Example 3: Input "123abc" (Invalid)

Step 1: Read '1' (digit, not letter)

Current State: START

Action: Cannot transition (no valid path)

Go to ERROR

Result: REJECTED (identifiers must start with letter)

7. Complete Example of Lexical Analysis

7.1 Example Code

Input Source Code:

```
while count < 10
```

7.2 Step-by-Step Token Recognition

Step 1: Process "while"

Read: w

Read: h

Read: i

Read: l

Read: e

Grouped: "while"

Pattern Match: Keyword pattern

Token Created: <KEYWORD_WHILE, ->

(Note: - means no additional attribute needed)

Step 2: Process whitespace

Read: ' ' (space)

Action: IGNORE (whitespace is discarded)

Continue to next character

Step 3: Process "count"

Read: c

Read: o

Read: u

Read: n

Read: t

Grouped: "count"

Pattern Match: Identifier pattern [a-zA-Z][a-zA-Z0-9]*

Token Created: <IDENTIFIER, count>

Step 4: Process whitespace

Read: ' ' (space)

Action: IGNORE

Continue to next character

Step 5: Process "<"

Read:

Grouped: "<"

Pattern Match: Operator pattern

Token Created: <OPERATOR_LT, ->

(LT means Less Than)

Step 6: Process whitespace

Read: ' ' (space)

Action: IGNORE

Continue to next character

Step 7: Process "10"

Read: 1

Read: 0

Grouped: "10"

Pattern Match: Number pattern [0-9]+

Token Created: <NUMBER, 10>

7.3 Final Token Stream

Complete Output:

TOKEN 1: <WHILE, ->

TOKEN 2: <IDENTIFIER, count>

TOKEN 3: <LT, ->

TOKEN 4: <NUMBER, 10>

This token stream is now passed to the Parser for syntax analysis.

IN SLIDE

A Simple Example of Lexical Analysis

while count < 10

Process:

Read **w h i l e** → matches KEYWORD_WHILE

Read **whitespace** → ignore

Read **c o u n t** → matches IDENTIFIER

Read **<** → matches OP_LT

Read **10** → matches NUMBER

Token stream:

<WHILE, -> <IDENTIFIER, count> <LT, -> <NUMBER, 10>

8. Error Handling in Lexical Analysis

8.1 What is a Lexical Error?

A lexical error occurs when the scanner encounters a character or sequence of characters that doesn't match any valid pattern.

8.2 Common Lexical Errors

Error Type 1: Invalid Character

Example: `x = 12 @ 3;`

Problem: '@' is not a valid character

Error Type 2: Malformed Number

Example: `12.34.56`

Problem: Multiple decimal points

Error Type 3: Invalid Identifier

Example: `123abc`

Problem: Identifier cannot start with digit

Error Type 4: Unclosed String

Example: `"hello world`

Problem: Missing closing quote

8.3 Error Reporting Example

Source Code with Error:

```
x = 12 @ 3;
```

Scanner Processing:

Token 1: <IDENTIFIER, x>

Token 2: <OPERATOR, =>

Token 3: <NUMBER, 12>

Token 4: ERROR!

Error Message:

"Lexical Error: Invalid symbol '@' at line 1, column 8"

Scanner Action:

- Report error
- Skip the invalid character
- Continue scanning
- Token 5: <NUMBER, 3>
- Token 6: <DELIMITER, ;>

8.4 Error Recovery Strategies

Strategy 1: Panic Mode

- Skip characters until a valid token is found
- Continue processing

Strategy 2: Delete Invalid Character

- Remove the invalid character
- Continue as if it wasn't there

Strategy 3: Insert Missing Character

- If a character seems missing, report it
- Example: Missing semicolon

Strategy 4: Replace Character

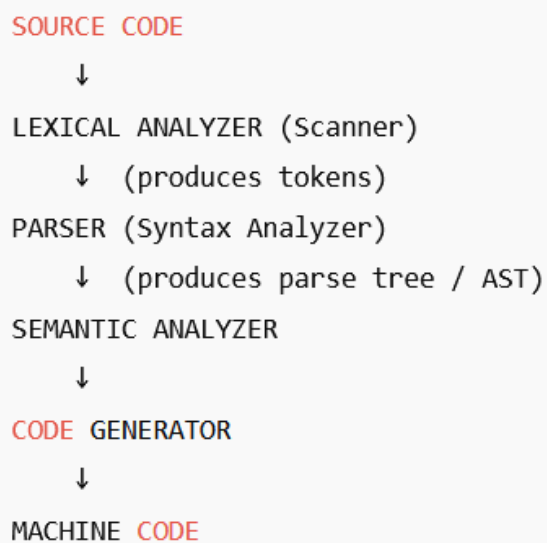
- Suggest correct character
- Example: "0" instead of "O" in numbers

9. Interaction Between Lexical Analyzer and Parser

9.1 Communication Flow

Overall Compilation Process:

SLIDE: Source Code → Lexical Analyzer → Tokens → Parser



9.2 How They Work Together

Method 1: Token-by-Token

The parser calls the lexical analyzer repeatedly:

Parser Request: "Give me next token"

↓

Scanner: Reads characters, identifies token

↓

Scanner Response: <IDENTIFIER, x>

↓

Parser: Processes token, requests next

↓

Scanner: Identifies next token

↓

Scanner Response: <OPERATOR, =>

↓

(This continues until end of file)

Pseudocode:

Function getNextToken():

 skip whitespace and comments

 read characters

 match against patterns

 return token

Parser calls:

 token1 = getNextToken() // Gets first token

 token2 = getNextToken() // Gets second token

 token3 = getNextToken() // Gets third token

 ... and so on

9.3 Key Points

Point 1: Parser never sees raw characters

- It only works with tokens
- Makes parsing simpler

Point 2: One token at a time

- Scanner provides tokens on demand
- Memory efficient approach

Point 3: Interface is clean

- Parser doesn't need to know about character processing
- Clear separation of concerns

Point 4: Error handling

- Lexical errors caught by scanner
- Syntax errors caught by parser

10. Token Categories

10.1 Common Token Types

Category 1: KEYWORDS

- Reserved words in the programming language
- Examples: if, else, while, for, int, float, return

Category 2: IDENTIFIERS

- Variable names, function names
- Pattern: letter (letter | digit | _)*
- Examples: x, myVar, count123, total_sum

Category 3: OPERATORS

- Arithmetic: +, -, *, /, %
- Relational: <, >, <=, >=, ==, !=
- Logical: &&, ||, !
- Assignment: =, +=, -=, *=, /=

Category 4: LITERALS/CONSTANTS

- Numbers: 123, 3.14, 0xFF
- Strings: "hello", "world"
- Characters: 'a', 'b'
- Boolean: true, false

Category 5: DELIMITERS

- Semicolon: ;
- Comma: ,
- Parentheses: ()
- Braces: { }
- Brackets: []

Category 6: COMMENTS

- Single-line: // comment
- Multi-line: /* comment */
- These are usually ignored (not converted to tokens)

10.2 Token Table Example

Source Code:

```
int sum = a + b;
```

Token Table:

Lexeme	Token Type	Attribute
int	KEYWORD	-
sum	IDENTIFIER	sum
=	OPERATOR	=
a	IDENTIFIER	a
+	OPERATOR	+
b	IDENTIFIER	b
;	DELIMITER	;

11. Tools for Lexical Analysis

11.1 Lex and Flex

What are they?

- **Lex:** Original lexical analyzer generator (Unix)
- **Flex:** Fast Lex, improved version of Lex

What do they do?

- Automatically generate lexical analyzer code
- Input: Specification file with patterns
- Output: C code for scanner

12. Summary of Key Concepts

12.1 Main Points to Remember

Point 1: Definition

- Lexical analysis is the first phase of compilation
- Converts character stream to token stream

Point 2: Core Components

- **Lexeme:** Actual text from source code
- **Token:** Category/classification of lexeme
- **Pattern:** Rule describing valid lexemes

Point 3: Internal Mechanism

- Uses Regular Expressions for patterns
- Implements patterns using Finite Automata (DFA)
- Processes input character by character

Point 4: Functions

- Removes whitespace and comments
- Groups characters into lexemes
- Produces tokens for parser
- Reports lexical errors

Point 5: Output

- Stream of tokens
- Each token has type and value
- Passed to parser for syntax analysis

12.2 Relationship Diagram

SOURCE CODE (characters)



[Scanner reads one character at a time]



[Groups characters into lexemes]



[Matches lexemes against patterns using DFA]



[Creates tokens from matched lexemes]



[Removes whitespace and comments]



TOKEN STREAM (tokens)



[Passed to Parser]

13.2 Sample Exam Questions

Question Type 1: Token Identification

Question:

Identify all tokens in: while (x < 10)

Answer:

Token 1: <KEYWORD, while>

Token 2: <DELIMITER, (>

Token 3: <IDENTIFIER, x>

Token 4: <OPERATOR, <>

Token 5: <NUMBER, 10>

Token 6: <DELIMITER, >

Question Type 2: Pattern Writing

Question:

Write a regular expression pattern for:

- a) Valid C identifiers
- b) Unsigned integers
- c) Floating point numbers

Answer:

a) `[a-zA-Z_][a-zA-Z0-9_]*`

b) `[0-9]+`

c) `[0-9]+\.[0-9]+`

Question Type 3: DFA Design

Question:

Design a DFA that recognizes the keyword "if"

Answer:

```
States: START, S1, S2, ACCEPT

START --i--> S1
S1 --f--> S2 (ACCEPT)
Any other transition --> ERROR

Text representation:
    i      f
START → S1 → S2 (ACCEPT)
```

Question Type 4: Error Detection

Question:

Identify lexical errors: `int 5x = @10;`

Answer:

Error 1: "5x" - Invalid identifier (cannot start with digit)

Error 2: "@" - Invalid character

Correct tokens:

- <KEYWORD, int>
- ERROR at "5x"
- <OPERATOR, =>
- ERROR at "@"
- <NUMBER, 10>
- <DELIMITER, ;>